CHUKA UNIVERSITY
CENTER FOR DATA ANALYTICS AND MODELING
Artificial Intelligence | Data Science | Consultancy | Research Hub

SESSION TWO: NUMERICAL & TABULAR COMPUTING WITH NUMPY AND PANDAS

D. K. MURIITHI

🔍 Main Goal

To **harness the power of Python libraries**, specifically **NumPy** and **Pandas**, for conducting efficient **numerical operations** and performing structured **data analysis**, enabling participants to manipulate and analyze real-world data effectively.

📌 Learning Outcomes

By the end of this session, participants will be able to:

✅ 1. Work with NumPy for Numerical Computation

- Create and manipulate **NumPy arrays**
- Understand and apply **slicing, indexing, and reshaping** techniques
- Utilize **vectorization** for performance optimization
- Apply **broadcasting** rules for array operations

✅ 2. Use Pandas for Tabular Data Manipulation

- Create and explore **Series and DataFrames**
- Perform essential operations: **filtering, sorting, renaming, and selecting data**
- Group and summarize data using **groupby**
- Merge and join multiple datasets effectively
- Reshape data using **pivot tables** and **melt**
- Handle **missing data** appropriately

✅ 3. Optimize Data Handling

- Perform **data type conversions** for better memory usage
- Understand **memory optimization techniques** for large datasets

🧪 Practical Application

Participants will:

- Load, explore, and clean real-world datasets (e.g., weather or sales data)

- Perform exploratory data analysis using NumPy and Pandas
- Apply learned techniques to reshape, summarize, and extract insights from data

Topics Covered

- NumPy arrays: creation, slicing, reshaping, broadcasting
- Vectorization for speed and efficiency
- Pandas DataFrames and Series operations
- Grouping, merging, pivoting, handling missing data
- Data type conversion and memory optimization

📊 NumPy Arrays: Creation, Slicing, Reshaping, and Broadcasting

📌 Introduction to NumPy Arrays

NumPy, short for *Numerical Python*, is a foundational Python library for numerical computing. It introduces the **ndarray** (N-dimensional array), a data structure designed for fast and efficient computation on large numerical datasets.

Advantages of NumPy arrays over native Python lists:

- Significantly faster for large datasets
- Occupies less memory
- Enables element-wise operations and advanced mathematical functions
- Supports broadcasting and vectorization, avoiding explicit loops

🧱 1. Creating NumPy Arrays

NumPy arrays can be created in various ways depending on the structure and initialization:

- **From Python Lists:** This is the simplest method, converting one-dimensional or nested Python lists into NumPy arrays.
- **Multidimensional Arrays:** Nested lists are converted into 2D or 3D arrays, useful for matrix-like data.
- **Special Arrays:** NumPy includes functions to create arrays filled with zeros, ones, or a constant value, as well as identity matrices.
- **Evenly Spaced Arrays:** Arrays can be generated using specified intervals (like ranges) or by dividing a range into equal parts.

These creation methods allow for efficient and structured data storage suited for numerical analysis.

✂️ 2. Slicing and Indexing

Once an array is created, specific elements or subarrays can be accessed and manipulated using **indexing and slicing**:

- **1D Indexing:** Single elements are accessed by their position.
- **2D Indexing:** Requires both row and column indices.
- **Slicing:** Allows retrieval of continuous subsets of data along specified dimensions.
- **Multidimensional Slicing:** Used to extract rows, columns, or even submatrices.

Key Note: Slicing in NumPy typically returns a *view* of the original data, meaning changes to the sliced array reflect in the original unless explicitly copied.

🔁 3. Reshaping Arrays

Reshaping allows for the transformation of the array's shape without altering its data. This is crucial when preparing data for machine learning or statistical analysis.

- **Reshape:** Changes the array's dimensions (e.g., from 1D to 2D or vice versa) as long as the total number of elements remains the same.
- **Flattening:** Converts any array to a one-dimensional array.
- **Transposing:** Swaps the array's rows and columns, which is especially important in matrix computations.

These operations enable flexible handling of data structures for computation, modeling, and visualization.

🔄 4. Broadcasting

Broadcasting is a powerful concept in NumPy that allows arithmetic operations between arrays of different shapes. It eliminates the need to manually replicate data to match shapes.

How broadcasting works:

- If two arrays have different shapes, NumPy automatically "stretches" the smaller array along dimensions where size is 1 or missing.
- This enables fast, element-wise operations between arrays without explicit loops.

Rules of Broadcasting:

1. If arrays have different dimensions, the one with fewer dimensions is padded with ones on the left.
2. Dimensions must either be equal or one of them must be 1 for broadcasting to occur.

Use Cases:

- Adding a scalar to every element of a matrix.
- Performing row- or column-wise operations without duplicating arrays.
- Combining arrays of different shapes in element-wise arithmetic operations.

🧠 Summary

Understanding how to create, manipulate, and transform NumPy arrays is foundational for data analysis and scientific computing. Here's a quick recap:

- **Creation:** NumPy offers multiple intuitive ways to initialize arrays.
- **Slicing and Indexing:** Provides efficient access to subsets of data.
- **Reshaping:** Enables converting arrays into desired forms for modeling and visualization.
- **Broadcasting:** Allows seamless operations between arrays of differing shapes, enhancing code efficiency and clarity.

Mastering these skills sets the groundwork for advanced data manipulation and machine learning tasks using Python.

In [1]:
```
import numpy as np

# Creating a NumPy array
arr = np.array([1, 2, 3, 4, 5])
print("Array:", arr)
```

```
Array: [1 2 3 4 5]
```

In [2]:
```
# Reshaping array
reshaped = arr.reshape(5, 1)
print("Reshaped to 5x1:\n", reshaped)
```

```
Reshaped to 5x1:
 [[1]
 [2]
 [3]
 [4]
 [5]]
```

In [3]:
```
# Slicing
print("First 3 elements:", arr[:3])
```

```
First 3 elements: [1 2 3]
```

In [8]:
```
# Broadcasting
arr2 = arr + 10
print("Broadcasted (add 10):", arr2)
```

```
Broadcasted (add 10): [11 12 13 14 15]
```

🧠 Vectorization for Speed and Efficiency

Vectorization is a core principle in **numerical computing** with Python, particularly when using libraries like **NumPy** and **Pandas**. It allows for concise, readable syntax and significant performance improvements by leveraging low-level optimizations and bypassing explicit Python loops.

🔹 What is Vectorization?

Definition: Vectorization refers to the process of replacing explicit loops with array expressions, enabling operations to be applied simultaneously to entire data structures (arrays or series), rather than processing elements one-by-one.

- Instead of iterating over values manually, vectorized code operates on whole collections at once.
- This results in both **cleaner syntax** and **faster execution**, since underlying implementations are often in compiled C code.

🔹 Why is Vectorization Important?

1. **Performance Boost**
    - Vectorized operations are **orders of magnitude faster** than traditional Python loops, especially with large datasets.
    - NumPy and Pandas use highly optimized C/C++ libraries under the hood.
2. **Readability**
    - Code becomes easier to understand and maintain.
    - Fewer lines, more intuitive expressions.
3. **Memory Efficiency**
    - Reduces overhead caused by Python's dynamic typing and function calls inside loops.
    - Memory is accessed in contiguous blocks, reducing cache misses.

🔹 Scalar vs Vectorized Operations

- Scalar operations: involve one value at a time.
- Vectorized operations: work with entire data structures (e.g., NumPy arrays or Pandas Series).

For example:

- A scalar operation might multiply each value in a list using a loop.
- A vectorized operation would multiply the entire array in one line.

🔹 Broadcasting and Vectorization

Broadcasting is a mechanism that allows NumPy to perform arithmetic operations on arrays of **different shapes**. This plays a critical role in vectorization.

- Enables efficient element-wise operations without needing explicit replication of data.
- Simplifies operations like scaling, centering, normalization, etc.

🔹 Avoiding Loops in Numerical Computations

Loops in Python:

- Are slower due to interpreter overhead.
- Don't leverage CPU-level optimizations.

Vectorized alternatives:

- Use NumPy/Pandas functions (e.g., `np.mean`, `df.sum()`).
- Replace list comprehensions with array expressions.
- Avoid `.apply()` unless necessary; prefer direct operations.

🔹 Functional Programming vs Vectorization

- Functional constructs like `map()`, `filter()`, and `reduce()` are improvements over loops.
- But they still fall short of the efficiency offered by vectorized operations in NumPy or Pandas.

In data science, **vectorization > functional programming > loops** in terms of performance.

🔹 Examples of Vectorized Use Cases (Conceptual)

1. **Mathematical Transformations**
    - Applying trigonometric or logarithmic functions to arrays.
    - Scaling and normalization.
2. **Statistical Computations**
    - Mean, median, standard deviation, variance computed on arrays directly.
    - Correlation matrices and covariance structures.
3. **Conditional Filtering**
    - Boolean indexing enables subsetting without loops.
    - Replaces multiple `if`/`else` statements in iteration.
4. **Element-wise Arithmetic**
    - Adding two columns or arrays together.
    - Multiplying or squaring arrays.

🔹 Pitfalls and Best Practices

Be cautious of:

- Implicit data type conversions (e.g., integer divided by float).

- Memory usage with large datasets (copying vs modifying in place).
- Using `.apply()` in Pandas where native vectorized methods exist.

Best Practices:

- Always prefer built-in vectorized functions over manual iterations.
- Profile your code (`%timeit`, `cProfile`, or `line_profiler`) to detect performance bottlenecks.
- Learn common NumPy/Pandas idioms for filtering, aggregation, and transformation.

🔹 Vectorization in Machine Learning Workflows

Machine learning pipelines benefit heavily from vectorized operations:

- Feature scaling (e.g., min-max normalization).
- Matrix operations for model training (e.g., dot products in linear models).
- Loss and gradient computations.

In frameworks like **scikit-learn**, **TensorFlow**, and **PyTorch**, vectorization is the standard due to its impact on training speed and efficiency.

🔹 Summary

Aspect	Loops	Vectorized Operations
Performance	Slow	Fast
Readability	Verbose	Concise
Scalability	Poor with large data	Efficient with big datasets
Ease of Debugging	Higher	Lower (but predictable)

📝 Final Thought

Vectorization isn't just a technique—it's a **paradigm shift** in how data operations are performed. Mastering vectorization is crucial for anyone working with large-scale data, enabling both **expressiveness** and **efficiency** in Python.

In [6]:
```
# Vectorization for performance
x = np.arange(1000000)
y = np.arange(1000000)

# Using vectorized operation
z = x + y
print("Vectorized sum completed.")
```

```
Vectorized sum completed.
```

The code above demonstrates **vectorization** in using NumPy, which is a fast and efficient way to perform operations on large arrays without using explicit loops.

```
# Vectorization for performance
x = np.arange(1000000)
y = np.arange(1000000)

# Using vectorized operation
z = x + y
print("Vectorized sum completed.")
```

Step-by-step Purpose

1. Create Large Arrays

```
x = np.arange(1000000)
y = np.arange(1000000)
```

- `np.arange(1000000)` creates a NumPy array containing numbers from `0` to `999999`.
- So:
  - `x = [0, 1, 2, 3, ..., 999999]`
  - `y = [0, 1, 2, 3, ..., 999999]`

These arrays each contain **1 million elements**.

---

2. Perform Vectorized Addition

```
z = x + y
```

This adds corresponding elements directly:

[z[i] = x[i] + y[i]]

Example:

x	y	z
0	0	0
1	1	2
2	2	4
3	3	6

So `z` becomes:

```
[0, 2, 4, 6, ..., 1999998]
```

Main Purpose of the Code

✅ Demonstrate Faster Computation

The code shows how NumPy performs operations on entire arrays at once instead of using slow Python loops.

Without Vectorization

You would normally do:

```
z = []
for i in range(len(x)):
    z.append(x[i] + y[i])
```

This is slower because Python loops have overhead.

Why Vectorization is Important

Benefits

- Faster execution
- Less code
- Better memory efficiency
- Uses optimized low-level C implementations internally

Performance Concept

This is commonly used in:

- Machine Learning
- Data Science
- Numerical Computing
- Deep Learning
- Scientific Simulations

because datasets can contain millions of values.

Mathematical Representation

The operation performed is:

$z_i = x_i + y_i$

for all elements $i = 0,1,2,\dots,999999$.

Final Output

The program only prints:

`Vectorized sum completed.`
But internally, the large addition operation has already been performed efficiently.

🧪 Example Task: Element-wise Multiplication

We want to multiply two arrays, `A` and `B`, each with **1 million elements**.

- Using a For Loop (Non-Vectorized)

In [3]:
```
import numpy as np
import time
```

In [4]:
```
# Create large arrays
A = np.random.rand(1000000)
B = np.random.rand(1000000)

# Non-vectorized multiplication using loop
result_loop = np.zeros(1000000)

start_time = time.time()
for i in range(len(A)):
    result_loop[i] = A[i] * B[i]
end_time = time.time()

print("Loop time:", end_time - start_time, "seconds")
```

```
Loop time: 0.6074340343475342 seconds
```

🔍 Explanation:

- `np.random.rand(n)` creates an array of `n` random numbers between 0 and 1.
- We preallocate an array `result_loop` to store the results.
- The `for` loop iterates through each index and multiplies `A[i] * B[i]`.
- We time the operation using `time.time()`.

2. ⚡ Using Vectorized Operation (NumPy)

Vectorized multiplication using NumPy

start_time = time.time() result_vectorized = A * B end_time = time.time()

```
print("Vectorized time:", end_time - start_time, "seconds")
```

🔍 Explanation:

- This single line `A * B` multiplies all corresponding elements **simultaneously**.
- NumPy handles this internally using optimized C loops and SIMD instructions.
- It's **faster and more readable**.

➡️ The vectorized version is **30x+ faster** than the looped one.

4. 💡 Additional Vectorized Operations

a. Add a constant to all elements

In [13]:
```
C = A + 5
### b. Apply a function (e.g., square root)
sqrt_A = np.sqrt(A)
```

In [15]:
```
### c. Logical comparison (e.g., filter values > 0.5)
greater_than_half = A[A > 0.5]
greater_than_half
```

Out[15]:
```
array([0.6498892 , 0.90999686, 0.65012544, ..., 0.60082497, 0.58792428,
       0.55974753])
```

📈 Summary of Benefits

Operation Type	For Loop (Manual)	NumPy Vectorized
Multiplication	`A[i] * B[i]` in loop	`A * B`
Add Constant	`A[i] + 5`	`A + 5`
Function Application	`math.sqrt(A[i])`	`np.sqrt(A)`
Filtering	`if A[i] > 0.5:`	`A[A > 0.5]`

📊 Pandas DataFrames and Series Operations

Overview

Pandas is a powerful open-source data analysis and manipulation library built on top of NumPy. It introduces two core data structures:

- **Series** – One-dimensional labeled array.
- **DataFrame** – Two-dimensional labeled data structure, similar to a table or spreadsheet.

These structures simplify data wrangling and analysis tasks.

1. 🧱 The Pandas Series

What is a Series?

A Series is a one-dimensional array-like structure that holds a sequence of values with associated labels (called the index).

Key Features:

- Supports operations like slicing, filtering, and mathematical functions.
- Index can be customized (e.g., integers, strings, dates).
- Automatically aligns data based on index during operations.

Common Use Cases:

- Representing a single column of data.
- Handling time series data.
- Quick statistical summaries on one-dimensional data.

2. 🧩 The Pandas DataFrame

What is a DataFrame?

A DataFrame is a two-dimensional data structure with rows and columns, each of which can hold different data types (int, float, string, etc.).

Key Features:

- Tabular format similar to SQL tables or Excel spreadsheets.
- Labeled axes (row index and column names).
- Handles missing values and heterogeneous data efficiently.
- Supports a wide variety of operations: filtering, aggregation, merging, reshaping, etc.

DataFrame Structure:

- **Index**: Labels for rows.
- **Columns**: Labels for columns.
- **Values**: Data contained in cells.

3. 🔎 Data Inspection and Basic Operations

Inspecting Structure and Contents:

- View column names, data types, dimensions, and index.
- Check for null values and summary statistics.

Selecting Data:

- Access single columns (returns Series).
- Select multiple columns or rows using labels or positions.
- Use slicing, indexing, and conditional filtering.

Modifying Data:

- Rename columns or index labels.
- Add or delete columns.
- Update values conditionally or via assignment.

4. 🔁 Arithmetic and Logical Operations

Element-wise Operations:

- Arithmetic between Series or DataFrames is automatically aligned by index and/or columns.
- Missing labels result in `NaN` unless explicitly handled.

Broadcasting:

- Scalars can be broadcast across an entire Series or DataFrame column.

Boolean Masking:

- Useful for subsetting data based on conditions.
- Produces a new object containing only rows that meet specified criteria.

5. 🧮 Aggregation and Summary Statistics

Common Aggregations:

- `sum()`, `mean()`, `median()`, `min()`, `max()`, `std()`, `var()`
- Can be applied to columns or rows.

Custom Aggregations:

- Use functions to perform more complex or tailored summaries.

- Apply row-wise or column-wise operations.

6. 🔄 Handling Missing Data

Types of Missing Data:

- Represented as `NaN` (Not a Number) or `None`.

Strategies to Handle:

- **Detection**: Use functions to identify missing values.
- **Removal**: Drop rows or columns containing nulls.
- **Imputation**: Fill missing values with mean, median, mode, or fixed values.

7. 🔗 Data Type Conversion

Type Inference:

- Pandas automatically infers data types (int, float, object, etc.).
- Sometimes manual type casting is required for analysis or memory optimization.

Conversion:

- Convert columns to appropriate types using casting functions.
- Useful in cleaning messy or mixed-type data columns.

8. 🔗 Combining Data

Concatenation:

- Stack multiple Series or DataFrames vertically or horizontally.

Merging (Join-like Operations):

- Combine datasets using keys (like SQL joins).
- Supports inner, outer, left, and right joins.

Appending:

- Add rows to an existing DataFrame.

9. 📐 Grouping and Aggregation

GroupBy Mechanism:

- Split data into groups based on categorical variables.
- Apply aggregations to each group separately.
- Combine the result into a single object.

Use Cases:

- Summarize sales by region.
- Analyze student performance by class.
- Average health metrics by gender.

10. 🔄 Reshaping and Pivoting Data

Reshape Techniques:

- **Pivot**: Rearrange data to change orientation of variables.
- **Melt**: Unpivot DataFrame from wide to long format.
- **Stack/Unstack**: Convert between hierarchical rows and columns.

Transpose:

- Switch rows and columns (for inspection or transformation).

11. 💾 Input and Output Operations

Reading Data:

- Load data from CSV, Excel, JSON, databases, and more.
- Custom parsing, headers, and encoding options.

Writing Data:

- Save processed data back to various formats.
- Export for sharing, storage, or reporting.

12. 📦 Memory and Performance Optimization

Reduce Memory Usage:

- Downcast data types where possible.
- Convert object columns with repetitive strings into categorical types.

Efficient Processing:

- Use vectorized operations over loops.
- Chain operations effectively for large-scale data.

13. 📘 Summary

Feature	Series	DataFrame
Dimensionality	1D	2D
Use Case	Single column or array	Structured tabular data
Operations	Element-wise, vectorized	Column-wise, row-wise operations
Real-World Analogy	A single column in Excel	A full Excel sheet

In [9]:
```
import pandas as pd

# Creating a DataFrame
data = {
    'Product': ['A', 'B', 'C', 'D'],
    'Sales': [100, 150, 200, 130],
    'Region': ['North', 'South', 'East', 'West']
}
data
```

Out[9]:
```
{'Product': ['A', 'B', 'C', 'D'],
 'Sales': [100, 150, 200, 130],
 'Region': ['North', 'South', 'East', 'West']}
```

In [10]:
```
df = pd.DataFrame(data)
df.head()
```

Out[10]:

	Product	Sales	Region
0	A	100	North
1	B	150	South
2	C	200	East
3	D	130	West

In [15]:
```
# Accessing Series
print("Sales column:", df['Sales'])
```

```
Sales column: 0    100
1    150
2    200
3    130
Name: Sales, dtype: int64
```

In [16]:
```
# Filtering
high_sales = df[df['Sales'] > 130]
high_sales
```

Out[16]:

	Product	Sales	Region
1	B	150	South
2	C	200	East

🔗 Grouping and Aggregation in Pandas

Grouping and aggregation are essential for data summarization, reporting, and exploratory data analysis. Pandas provides powerful tools to group data based on specific criteria and apply aggregate functions to each group.

1. ✅ What is Grouping?

Grouping refers to the process of:

1. **Splitting** the data into groups based on some criteria (e.g., category, time period).
2. **Applying** a function to each group independently (e.g., sum, mean, count).
3. **Combining** the results into a DataFrame or Series.

This is commonly referred to as the **Split-Apply-Combine** strategy.

2. 🎯 Why Use Grouping?

Grouping is useful when you want to:

- Compute summary statistics for subgroups.
- Perform comparisons across categories.
- Identify patterns in segmented data.
- Prepare data for visualization or modeling.

3. 🧩 Common Grouping Keys

You can group data using:

- A single column (e.g., `Gender`, `Region`)
- Multiple columns (e.g., `Department` and `Year`)
- A function or custom mapping (e.g., grouping by age bins or date parts)

4. 📊 Aggregation Functions

Built-in Aggregation Functions:

- `sum()` – Total of values
- `mean()` – Average value
- `median()` – Middle value
- `count()` – Number of non-null values
- `min()` / `max()` – Minimum or maximum
- `std()` / `var()` – Standard deviation and variance

These functions can be applied after grouping to summarize the data.

5. 🧠 Custom Aggregations

You can define custom aggregation logic for more complex summaries. Examples include:

- Conditional aggregation (e.g., average only where values > 50)
- Applying different functions to different columns
- Aggregating based on external rules or transformations

6. 📚 Multi-level Grouping

Grouping by multiple columns creates a **MultiIndex** (hierarchical index). This is useful for:

- Breaking down analysis by more than one category
- Creating multi-dimensional summaries (e.g., sales by region and quarter)

7. 🌀 Transform vs. Aggregate vs. Filter

- **Aggregate**: Summarizes each group into a single row.
- **Transform**: Returns a result the same shape as the input (e.g., standardizing within each group).
- **Filter**: Removes entire groups based on a condition (e.g., keep groups with more than 10 members).

8. 🪄 Chaining Groupby with Other Methods

Groupby results can be further modified using:

- **Sorting**: Order groups by aggregated values.
- **Resetting index**: Convert grouped index back to regular columns.
- **Pivoting**: Reshape grouped data for reporting and visualization.

9. 📓 Use Cases in Real-World Data

- **Finance**: Total revenue by quarter and department
- **Education**: Average test scores by class and subject
- **Healthcare**: Number of cases by region and age group
- **Retail**: Customer purchase frequency by segment
- **Environment**: Average temperature by month and location

10. 🧮 Summary Table of Grouping Functions

Task	Function Type	Description
Group by one column	`.groupby('col')`	Basic grouping
Group by multiple columns	`.groupby(['a','b'])`	Hierarchical grouping
Apply single function	`.mean()`	Compute average per group
Apply multiple functions	`.agg(['mean','sum'])`	Multiple aggregations
Custom function	`.agg(custom_func)`	User-defined aggregation logic
Reshape output	`.unstack()` / `.pivot_table()`	For cleaner summaries

In [22]:
```
# Grouping and aggregation
data = {
    'Region': ['North', 'North', 'South', 'South'],
    'Sales': [100, 150, 80, 90]
}
df = pd.DataFrame(data)
df.head()
```

Out[22]:

	Region	Sales
0	North	100
1	North	150
2	South	80
3	South	90

Summarized Region [Mean]

In [25]:
```
grouped = df.groupby('Region').mean()
grouped
```

Out[25]:

Region	Sales
North	125.0
South	85.0

🔗 Merging Data in Pandas

Data often exists across multiple tables or files. Merging (also known as **joining**) allows us to combine this data into a unified dataset based on common fields (keys). This is essential in real-world data analysis where we bring together related information.

1. ✅ What is Data Merging?

Merging refers to the process of combining two or more DataFrames into one, using a common key (or set of keys). This is similar to SQL joins. It enables analysts to enrich datasets by pulling in additional columns or observations from other sources.

2. 🔍 Key Concepts in Merging

a. **Keys**

Keys are columns used to align the datasets. They are the basis for determining which rows match across the DataFrames.

- **Primary key**: Unique identifier in one table.
- **Foreign key**: Key used to link to another table's primary key.

b. **Join Types**

Pandas supports different types of joins:

- **Inner Join**: Returns rows with matching keys in both tables.
- **Left Join**: Keeps all rows from the left DataFrame and adds matching rows from the right DataFrame.
- **Right Join**: Keeps all rows from the right DataFrame and adds matching rows from the left DataFrame.
- **Outer Join** (Full Join): Keeps all rows from both DataFrames, fills missing values where no match exists.

3. 🧩 When to Use Merging

- Combining customer data with transaction history
- Linking survey responses with demographic information
- Merging sales records with product catalogs
- Joining geographic codes with population data

4. 🗃️ Types of Merging Scenarios

a. **One-to-One Merge**

Each key exists only once in both DataFrames.

b. **Many-to-One Merge**

One DataFrame has duplicate key values; the other has unique key values. Common in merging transactions with user metadata.

c. **Many-to-Many Merge**

Both DataFrames have duplicate keys. This results in a Cartesian product for matching keys (can increase row count significantly).

5. 🧠 Important Merge Parameters

- **on**: Specifies the column(s) to merge on (must be present in both DataFrames).
- **left_on / right_on**: When keys have different names in each DataFrame.
- **how**: Specifies type of join (`'inner'` , `'left'` , `'right'` , `'outer'`).
- **suffixes**: Handles overlapping column names from both DataFrames by appending suffixes like `_x` and `_y` .

6. 🧮 Handling Duplicates and Conflicts

When merging:

- If the key columns contain duplicates, the resulting DataFrame may have multiple matching rows.
- Column name collisions (same column names from both DataFrames) are resolved with suffixes.
- Inconsistent or missing key values will result in `NaN` for unmatched records.

7. 📝 Practical Considerations

a. **Data Consistency**

Ensure that merge keys are of the same data type and cleaned (no trailing spaces or inconsistent formats).

b. **Missing Values**

Choose join type carefully to avoid unintentional loss of data due to non-matching keys.

c. **Performance**

Merging large datasets can be memory intensive. Consider indexing, filtering, or chunk processing for large-scale merges.

8. 🔄 Difference Between `merge`, `join`, and `concat`

- `merge()` : Flexible and SQL-style joins on keys.
- `join()` : Simplified syntax for merging based on index or a key.
- `concat()` : Used for stacking datasets vertically (adding rows) or horizontally (adding columns), not strictly for joining on keys.

9. 📚 Real-World Examples

Use Case	Description
Healthcare Data	Merge patient records with hospital visits and lab results
Education	Join student performance with demographic and enrollment data
Finance	Combine stock price data with financial ratios and sectors
Survey Analysis	Link responses to region, age group, and income segments
Retail	Merge order details with product descriptions and pricing

10. 📝 Summary Table

Join Type	Keeps Non-Matching	Description
Inner Join	❌ No	Only matching rows from both tables
Left Join	✅ Left only	All left rows + matched right rows
Right Join	✅ Right only	All right rows + matched left rows
Outer Join	✅ All	All rows from both, fill missing data

Merging is a foundational tool in data wrangling. Understanding how to use it correctly empowers analysts to work across multiple data sources confidently and efficiently.

Here are examples of various join operations using two DataFrames (`df_left` and `df_right`) with 6 variables and 20 observations:

✅ `df_left` – Sample (First 5 Rows)

ID	Name	Age	Gender	City	Score
1	Name_1	28	Female	Kisumu	75
2	Name_2	48	Female	Nairobi	84
3	Name_3	34	Male	Mombasa	89
4	Name_4	29	Female	Kisumu	97
5	Name_5	27	Female	Nairobi	96

✅ `df_right` – Sample (First 5 Rows)

ID	Department	Salary	Tenure	Supervisor	Location
15	IT	74067	7	Sup_1	Remote
16	Admin	63026	8	Sup_2	Remote
17	HR	92742	5	Sup_3	HQ
18	HR	80437	3	Sup_4	HQ
19	Finance	98731	3	Sup_5	HQ

🔁 Inner Join

- Combines only the rows with matching `ID` s in both DataFrames.
- Only IDs from 15 to 20 appear.

🔁 Left Join

- All rows from `df_left` are preserved.
- Matching rows from `df_right` are included.
- Non-matching rows from `df_right` will show `NaN` for right-side columns.

🔁 Right Join

- All rows from `df_right` are preserved.
- Matching rows from `df_left` are included.

- Non-matching rows from `df_left` will show `NaN` for left-side columns.

🔁 Outer Join

- All rows from both DataFrames are included.
- Non-matching rows will show `NaN` on the side where there's no match.

In [17]:
```
import pandas as pd
import numpy as np

# Sample Data for df_left
np.random.seed(1)  # For reproducibility
df_left = pd.DataFrame({
    'ID': range(1, 21),  # IDs 1 to 20
    'Name': [f'Name_{i}' for i in range(1, 21)],
    'Age': np.random.randint(25, 50, size=20),
    'Gender': np.random.choice(['Male', 'Female'], size=20),
    'City': np.random.choice(['Nairobi', 'Mombasa', 'Kisumu'], size=20),
    'Score': np.random.randint(60, 100, size=20)
})
df_left
```

Out[17]:

	ID	Name	Age	Gender	City	Score
0	1	Name_1	30	Female	Mombasa	72
1	2	Name_2	36	Male	Kisumu	86
2	3	Name_3	37	Male	Mombasa	76
3	4	Name_4	33	Male	Mombasa	78
4	5	Name_5	34	Female	Nairobi	75
5	6	Name_6	36	Male	Nairobi	60
6	7	Name_7	30	Male	Mombasa	64
7	8	Name_8	40	Male	Nairobi	85
8	9	Name_9	25	Female	Nairobi	94
9	10	Name_10	41	Female	Mombasa	83
10	11	Name_11	26	Female	Kisumu	67
11	12	Name_12	37	Female	Mombasa	86
12	13	Name_13	32	Female	Nairobi	85
13	14	Name_14	38	Male	Kisumu	82
14	15	Name_15	31	Male	Kisumu	69
15	16	Name_16	43	Male	Mombasa	63
16	17	Name_17	45	Female	Mombasa	99
17	18	Name_18	30	Female	Mombasa	83
18	19	Name_19	43	Female	Nairobi	96
19	20	Name_20	45	Female	Nairobi	87

In [18]:

```
# Sample Data for df_right
df_right = pd.DataFrame({
    'ID': range(15, 35),  # IDs 15 to 34
    'Department': np.random.choice(['IT', 'HR', 'Finance', 'Admin'], size=20),
    'Salary': np.random.randint(60000, 100000, size=20),
    'Tenure': np.random.randint(1, 10, size=20),
    'Supervisor': [f'Sup_{i}' for i in range(1, 21)],
    'Location': np.random.choice(['Remote', 'HQ'], size=20)
})
df_right
```

Out[18]:

	ID	Department	Salary	Tenure	Supervisor	Location
0	15	HR	98616	5	Sup_1	HQ
1	16	HR	65271	1	Sup_2	Remote
2	17	Admin	97559	3	Sup_3	Remote
3	18	Finance	69537	1	Sup_4	Remote
4	19	IT	73681	8	Sup_5	Remote
5	20	IT	70061	2	Sup_6	HQ
6	21	Finance	93229	8	Sup_7	HQ
7	22	Finance	89236	9	Sup_8	Remote
8	23	Admin	64059	5	Sup_9	Remote
9	24	Admin	90869	1	Sup_10	HQ
10	25	Admin	93483	2	Sup_11	HQ
11	26	Admin	82791	9	Sup_12	HQ
12	27	HR	66221	3	Sup_13	Remote
13	28	Admin	95528	4	Sup_14	Remote
14	29	Admin	61067	2	Sup_15	Remote
15	30	IT	72820	3	Sup_16	Remote
16	31	Finance	84094	8	Sup_17	Remote
17	32	Finance	69764	3	Sup_18	HQ
18	33	Finance	73159	7	Sup_19	HQ
19	34	IT	63885	1	Sup_20	HQ

In [20]:
```
# 🔄 INNER JOIN
inner_join = pd.merge(df_left, df_right, on='ID', how='inner')
inner_join
```

Out[20]:

	ID	Name	Age	Gender	City	Score	Department	Salary	Tenure	Supervisor
0	15	Name_15	31	Male	Kisumu	69	HR	98616	5	Sup_1
1	16	Name_16	43	Male	Mombasa	63	HR	65271	1	Sup_2
2	17	Name_17	45	Female	Mombasa	99	Admin	97559	3	Sup_3
3	18	Name_18	30	Female	Mombasa	83	Finance	69537	1	Sup_4
4	19	Name_19	43	Female	Nairobi	96	IT	73681	8	Sup_5
5	20	Name_20	45	Female	Nairobi	87	IT	70061	2	Sup_6

In [21]:

```
# 🔁 LEFT JOIN
left_join = pd.merge(df_left, df_right, on='ID', how='left')
left_join
```

Out[21]:

	ID	Name	Age	Gender	City	Score	Department	Salary	Tenure	Superviso
0	1	Name_1	30	Female	Mombasa	72	NaN	NaN	NaN	NaN
1	2	Name_2	36	Male	Kisumu	86	NaN	NaN	NaN	NaN
2	3	Name_3	37	Male	Mombasa	76	NaN	NaN	NaN	NaN
3	4	Name_4	33	Male	Mombasa	78	NaN	NaN	NaN	NaN
4	5	Name_5	34	Female	Nairobi	75	NaN	NaN	NaN	NaN
5	6	Name_6	36	Male	Nairobi	60	NaN	NaN	NaN	NaN
6	7	Name_7	30	Male	Mombasa	64	NaN	NaN	NaN	NaN
7	8	Name_8	40	Male	Nairobi	85	NaN	NaN	NaN	NaN
8	9	Name_9	25	Female	Nairobi	94	NaN	NaN	NaN	NaN
9	10	Name_10	41	Female	Mombasa	83	NaN	NaN	NaN	NaN
10	11	Name_11	26	Female	Kisumu	67	NaN	NaN	NaN	NaN
11	12	Name_12	37	Female	Mombasa	86	NaN	NaN	NaN	NaN
12	13	Name_13	32	Female	Nairobi	85	NaN	NaN	NaN	NaN
13	14	Name_14	38	Male	Kisumu	82	NaN	NaN	NaN	NaN
14	15	Name_15	31	Male	Kisumu	69	HR	98616.0	5.0	Sup_
15	16	Name_16	43	Male	Mombasa	63	HR	65271.0	1.0	Sup_
16	17	Name_17	45	Female	Mombasa	99	Admin	97559.0	3.0	Sup_
17	18	Name_18	30	Female	Mombasa	83	Finance	69537.0	1.0	Sup_
18	19	Name_19	43	Female	Nairobi	96	IT	73681.0	8.0	Sup_
19	20	Name_20	45	Female	Nairobi	87	IT	70061.0	2.0	Sup_

In [22]:

```
# 🔁 RIGHT JOIN
right_join = pd.merge(df_left, df_right, on='ID', how='right')
right_join
```

Out[22]:

	ID	Name	Age	Gender	City	Score	Department	Salary	Tenure	Supervisor
0	15	Name_15	31.0	Male	Kisumu	69.0	HR	98616	5	Sup_1
1	16	Name_16	43.0	Male	Mombasa	63.0	HR	65271	1	Sup_2
2	17	Name_17	45.0	Female	Mombasa	99.0	Admin	97559	3	Sup_3
3	18	Name_18	30.0	Female	Mombasa	83.0	Finance	69537	1	Sup_4
4	19	Name_19	43.0	Female	Nairobi	96.0	IT	73681	8	Sup_5
5	20	Name_20	45.0	Female	Nairobi	87.0	IT	70061	2	Sup_6
6	21	NaN	NaN	NaN	NaN	NaN	Finance	93229	8	Sup_7
7	22	NaN	NaN	NaN	NaN	NaN	Finance	89236	9	Sup_8
8	23	NaN	NaN	NaN	NaN	NaN	Admin	64059	5	Sup_9
9	24	NaN	NaN	NaN	NaN	NaN	Admin	90869	1	Sup_10
10	25	NaN	NaN	NaN	NaN	NaN	Admin	93483	2	Sup_11
11	26	NaN	NaN	NaN	NaN	NaN	Admin	82791	9	Sup_12
12	27	NaN	NaN	NaN	NaN	NaN	HR	66221	3	Sup_13
13	28	NaN	NaN	NaN	NaN	NaN	Admin	95528	4	Sup_14
14	29	NaN	NaN	NaN	NaN	NaN	Admin	61067	2	Sup_15
15	30	NaN	NaN	NaN	NaN	NaN	IT	72820	3	Sup_16
16	31	NaN	NaN	NaN	NaN	NaN	Finance	84094	8	Sup_17
17	32	NaN	NaN	NaN	NaN	NaN	Finance	69764	3	Sup_18
18	33	NaN	NaN	NaN	NaN	NaN	Finance	73159	7	Sup_19
19	34	NaN	NaN	NaN	NaN	NaN	IT	63885	1	Sup_20

In [23]:

```
# 🔁 OUTER JOIN
outer_join = pd.merge(df_left, df_right, on='ID', how='outer')
outer_join
```

Out[23]:

	ID	Name	Age	Gender	City	Score	Department	Salary	Tenure	Supervisc
0	1	Name_1	30.0	Female	Mombasa	72.0	NaN	NaN	NaN	Nal
1	2	Name_2	36.0	Male	Kisumu	86.0	NaN	NaN	NaN	Nal
2	3	Name_3	37.0	Male	Mombasa	76.0	NaN	NaN	NaN	Nal
3	4	Name_4	33.0	Male	Mombasa	78.0	NaN	NaN	NaN	Nal
4	5	Name_5	34.0	Female	Nairobi	75.0	NaN	NaN	NaN	Nal
5	6	Name_6	36.0	Male	Nairobi	60.0	NaN	NaN	NaN	Nal
6	7	Name_7	30.0	Male	Mombasa	64.0	NaN	NaN	NaN	Nal
7	8	Name_8	40.0	Male	Nairobi	85.0	NaN	NaN	NaN	Nal
8	9	Name_9	25.0	Female	Nairobi	94.0	NaN	NaN	NaN	Nal
9	10	Name_10	41.0	Female	Mombasa	83.0	NaN	NaN	NaN	Nal
10	11	Name_11	26.0	Female	Kisumu	67.0	NaN	NaN	NaN	Nal
11	12	Name_12	37.0	Female	Mombasa	86.0	NaN	NaN	NaN	Nal
12	13	Name_13	32.0	Female	Nairobi	85.0	NaN	NaN	NaN	Nal
13	14	Name_14	38.0	Male	Kisumu	82.0	NaN	NaN	NaN	Nal
14	15	Name_15	31.0	Male	Kisumu	69.0	HR	98616.0	5.0	Sup_
15	16	Name_16	43.0	Male	Mombasa	63.0	HR	65271.0	1.0	Sup_
16	17	Name_17	45.0	Female	Mombasa	99.0	Admin	97559.0	3.0	Sup_
17	18	Name_18	30.0	Female	Mombasa	83.0	Finance	69537.0	1.0	Sup_
18	19	Name_19	43.0	Female	Nairobi	96.0	IT	73681.0	8.0	Sup_
19	20	Name_20	45.0	Female	Nairobi	87.0	IT	70061.0	2.0	Sup_
20	21	NaN	NaN	NaN	NaN	NaN	Finance	93229.0	8.0	Sup_
21	22	NaN	NaN	NaN	NaN	NaN	Finance	89236.0	9.0	Sup_
22	23	NaN	NaN	NaN	NaN	NaN	Admin	64059.0	5.0	Sup_
23	24	NaN	NaN	NaN	NaN	NaN	Admin	90869.0	1.0	Sup_1
24	25	NaN	NaN	NaN	NaN	NaN	Admin	93483.0	2.0	Sup_1
25	26	NaN	NaN	NaN	NaN	NaN	Admin	82791.0	9.0	Sup_1
26	27	NaN	NaN	NaN	NaN	NaN	HR	66221.0	3.0	Sup_1
27	28	NaN	NaN	NaN	NaN	NaN	Admin	95528.0	4.0	Sup_1
28	29	NaN	NaN	NaN	NaN	NaN	Admin	61067.0	2.0	Sup_1
29	30	NaN	NaN	NaN	NaN	NaN	IT	72820.0	3.0	Sup_1

	ID	Name	Age	Gender	City	Score	Department	Salary	Tenure	Supervisc
30	31	NaN	NaN	NaN	NaN	NaN	Finance	84094.0	8.0	Sup_1
31	32	NaN	NaN	NaN	NaN	NaN	Finance	69764.0	3.0	Sup_1
32	33	NaN	NaN	NaN	NaN	NaN	Finance	73159.0	7.0	Sup_1
33	34	NaN	NaN	NaN	NaN	NaN	IT	63885.0	1.0	Sup_2

In [33]:
```
# 🔁 LEFT SEMI JOIN (only rows in df_left with matching ID in df_right)
left_semi_join = df_left[df_left['ID'].isin(df_right['ID'])]
left_semi_join.head()
```

Out[33]:

	ID	Name	Age	Gender	City	Score
14	15	Name_15	31	Male	Kisumu	69
15	16	Name_16	43	Male	Mombasa	63
16	17	Name_17	45	Female	Mombasa	99
17	18	Name_18	30	Female	Mombasa	83
18	19	Name_19	43	Female	Nairobi	96

In [34]:
```
# 🔁 LEFT ANTI JOIN (only rows in df_left with no matching ID in df_right)
left_anti_join = df_left[~df_left['ID'].isin(df_right['ID'])]
left_anti_join.head()
```

Out[34]:

	ID	Name	Age	Gender	City	Score
0	1	Name_1	30	Female	Mombasa	72
1	2	Name_2	36	Male	Kisumu	86
2	3	Name_3	37	Male	Mombasa	76
3	4	Name_4	33	Male	Mombasa	78
4	5	Name_5	34	Female	Nairobi	75

✅ Summary of Join Results:

Join Type	Description
`inner`	Matching rows on both sides (IDs 15–20)
`left`	All rows from `df_left` + matched from `df_right`
`right`	All rows from `df_right` + matched from `df_left`
`outer`	All rows from both `df_left` and `df_right`

Join Type	Description
`left_semi`	Rows from `df_left` where `ID` exists in `df_right`, only `df_left` columns
`left_anti`	Rows from `df_left` where `ID` **does not exist** in `df_right`

🧠 1. Why Data Type Conversion and Memory Optimization Matter

- Large datasets can consume a lot of memory and slow down processing.
- Proper data type conversion can **reduce memory usage**, **speed up computation**, and **optimize storage**.
- Especially crucial in **data preprocessing pipelines** and **production systems**.

🏷️ 2. Common Data Types in Pandas

Data Type	Description	Typical Memory Size
`int64`	64-bit integer	8 bytes
`float64`	64-bit floating point number	8 bytes
`object`	Mixed types, mostly strings	Variable
`category`	Categorical (coded as integers)	1–2 bytes
`datetime64`	Date and time values	8 bytes
`bool`	Boolean (`True` / `False`)	1 byte

🔁 3. Common Type Conversion Scenarios

🔹 Numeric Downcasting

- Convert `int64` → `int32` or `int16` where possible.
- Convert `float64` → `float32` if precision can be sacrificed slightly.

🔹 Object to Category

- Convert string columns with **low cardinality** (few unique values) to `category` type.
- This saves memory and speeds up grouping, sorting, and merging.

🔹 Object to DateTime

- Strings representing dates should be converted to `datetime64`.
- Enables time-based indexing, resampling, and efficient manipulation.

🔹 Object to Boolean

- If a column contains only two values (e.g., 'Yes', 'No'), it can be mapped to `bool`.

💾 4. Techniques for Memory Optimization

✅ Downcast Numeric Columns

- Use downcasting when values don't need full 64-bit range.

✅ Use Categorical for Repeated Strings

- Reduces memory drastically by storing a **dictionary of unique values** with integer codes.

✅ Drop Unnecessary Columns

- Remove columns not used in analysis or modeling.

✅ Optimize Data During Reading

- Specify `dtype` for each column when loading large datasets (e.g., in `read_csv()`).
- Parse dates directly while reading.

✅ Use Sparse Data Structures

- For data with many zeros or NaNs (e.g., one-hot encoded matrices), use sparse formats.

🧮 5. Evaluating Memory Usage

- Use functions to **inspect memory consumption** of your DataFrame.
- Helps identify which columns take up the most space and need optimization.

⚠️ 6. Caveats and Best Practices

- **Precision Loss**: Downcasting floats may cause rounding errors.
- **Performance vs Memory Trade-Off**: More compact types may slightly slow down computations in some cases.
- **Check for Nulls**: Some conversions (e.g., to `int`) require handling of missing values.
- **Categorical Pitfalls**: Don't convert high-cardinality strings to categories unnecessarily — it can increase memory usage.

🔍 Code Breakdown and Explanation

In [39]:
```
df = pd.DataFrame({
    'int_col': range(1000),
    'float_col': np.random.rand(1000)
})
df.head()
```

Out[39]:

	int_col	float_col
0	0	0.434140
1	1	0.179209
2	2	0.650436
3	3	0.913288
4	4	0.051405

- Creates a DataFrame `df` with 1000 rows and two columns:
  - `int_col` : integers from 0 to 999 (default dtype: `int64`)
  - `float_col` : 1000 random float numbers between 0 and 1 (default dtype: `float64`)

In [40]:
```
print("Original dtypes:", df.dtypes)
```

```
Original dtypes: int_col        int64
float_col    float64
dtype: object
```

- Prints the original data types of the two columns:

In [42]:
```
df['int_col'] = df['int_col'].astype(np.int16)
df['float_col'] = df['float_col'].astype(np.float32)
```

- **Downcasts** the numeric columns to smaller, more memory-efficient types:
  - `int_col` : from `int64` → `int16` (can store values from -32,768 to 32,767)
  - `float_col` : from `float64` → `float32` (lower precision, less memory)

In [43]:
```
print("Optimized dtypes:", df.dtypes)
```

```
Optimized dtypes: int_col        int16
float_col    float32
dtype: object
```

- Shows the new, optimized data types:

✅ Why This Matters

- `int64` and `float64` take **8 bytes per value**.
- `int16` and `float32` take only **2 bytes and 4 bytes**, respectively.
- For large datasets, this change can save **significant memory** without affecting performance or accuracy (when precision is not critical).

📌 Summary

This code:

- Creates a sample DataFrame.
- Prints the original data types.
- Converts the data types to more memory-efficient versions.
- Prints the updated data types.

This is a simple but powerful technique to **optimize memory usage in data preprocessing**.

In [ ]: